

배열 완성 과제에서 코드 생성 모델의 선호와 예측 단서 분석: 리액트 훅의 함수-배열 구조를 중심으로

박준영

인문대학 언어학과

서울대학교

bloomwayz@snu.ac.kr

초록

대형 언어 모델은 코드 생성 과제에서 높은 성능을 보이고 있으며, 이에 따라 코드 모델이 어떤 정보를 표상하고 어떤 표면적 단서에 의존하는지를 분석하는 연구가 이어져 왔다. 그러나 모델의 다음 토큰 예측이 어떤 국소적인 단서에 따라 달라지는지는 아직 충분히 분석되지 않았다. 이 연구는 리액트 훅 `useEffect`와 유사한 함수-배열 구조에서, 모델이 배열의 첫 항목으로 상태 변수와 상수 가운데 어느 쪽을 선호하는지 분석한다. 이를 위해 우리는 45개의 변수 쌍을 바탕으로 자바스크립트 코드 데이터셋을 구축하고, Llama 3.2 1B와 Gemma 3 1B 모델이 배열의 첫 항목 위치에서 두 후보 변수에 부여하는 로짓을 비교했다. 또한 함수 본문에서의 변수 사용과 문맥을 조작하는 소거 실험을 통해 관찰된 선호를 분해했다. 선호 관찰 실험에서 두 모델은 모든 함수 이름 조건에 걸쳐 상태 변수를 선호했으며, 이 선호는 리액트 환경보다 일반 자바스크립트 환경에서 오히려 더 크게 나타났다. 변수 사용 소거 실험에서는 `useState` 형태의 선언에서 오는 선언 형태 단서와 함수 본문에서 어떤 변수가 사용되었는지에 관한 변수 사용 단서가 함께 작동하며, 두 단서의 상대적 우세가 배열을 도입하는 문맥에 따라 달라짐을 보였다. 문맥 소거 실험에서는 이 선호가 의존성 배열이나 함수-배열 구조에 국한되지 않고, 독립적인 배열 완성 문맥에서도 양의 방향으로 유지됨을 확인했다. 이러한 결과는 모델의 상태 변수 선호가 배열 완성 과정에서 선언 형태 단서와 변수 사용 단서가 결합해 나타나는 경향임을 시사한다. 이 실험에 사용한 데이터셋 및 코드는 <https://github.com/young-52/array-behaviour>에 공개되어 있다.

1 서론

대형 언어 모델 *large language models (LLMs)*의 프로그래밍 성능은 계속해서 향상되어 왔으며, 최근 모델

은 코드 생성 작업에서 강한 성능을 보인다(Austin et al., 2021; Fried et al., 2023). 이러한 성능에 힘입어 인공지능 도구는 이미 개발 현장에 폭넓게 자리 잡아 대부분의 개발자가 인공지능 도구를 개발에 활용하고 있고, 그중 많은 이들이 인공지능 도구의 성능에 호의적이라고 밝혔다(Stack Exchange, Inc., 2025).

이 추세를 따라 코드 생성 모델의 행동과 내부 구조에 대한 연구도 다방면으로 이루어졌다. 대표적으로 모델이 코드의 문법 구조, 식별자, 데이터 흐름, 제어 흐름, 기능적 동등성과 같은 정보를 어느 정도 표상하는지 분석한 연구가 있고(Troshin and Chirkova, 2022; Ma et al., 2024; Maveli et al., 2025), 코드 모델의 예측이 변수명이나 의미 보존 변형과 같은 표면적 변화에 민감할 수 있음을 보인 연구도 있다(Yefet et al., 2020; Wang et al., 2024; Orvalho and Kwiatkowska, 2025). 이러한 연구들은 코드 모델의 예측이 프로그램 의미뿐 아니라 다양한 표면적·구조적 단서의 영향을 받을 수 있음을 시사한다.

다만 이들 연구는 주로 완성된 코드의 정답 여부나 오류 유형, 의미 보존 변형에 대한 강건성, 모델 표상의 전반적인 성질을 분석하는 데 초점을 두었다. 특정 위치에서 후보 토큰의 로짓이 어떤 단서에 따라 달라지는지는 아직 충분히 분석되지 않았으며, 특히 라이브러리의 의미 규약과 일반적인 문법 패턴이 함께 작동하는 상황에서 모델이 다음 토큰을 예측할 때 어떤 단서에 기대는지를 국소적으로 살펴보는 연구는 찾아보기 어렵다.

이에, 이 글은 대형 언어 모델이 코드의 다음 토큰을 예측할 때 어떤 토큰을 선호하는지 관찰하고, 이때 어떤 정보가 어느 정도로 코드 생성에 영향을 미치는지 분석한다. 특히 이 연구에서는 리액트 *React* 라이브러리에서 제공되는 함수 가운데 하나인 `useEffect`의 구조와 유사한 함수-배열 패턴에 모델이 어떻게

반응하는지를 집중적으로 살펴본다. 아래는 우리가 연구에서 사용한 함수-배열 패턴의 예시이다.

```
1 const [s, setS] = useState(0);
2 const c = 0;
3
4 useEffect(() => {
5   fetch(`/api/me?x=${s}&y=${c}`)
6 }, [s])
```

우선 위 코드에 등장하는 변수를 살펴보자. 첫 번째 줄에서는 상태 변수 `s`와 상태를 갱신하는 함수 `setS`를 선언하고 있고, 두 번째 줄에서는 상수 `c`를 선언하고 있다. 상태 `s`와 상수 `c`는 얼핏 닮아 보이지만, 상태는 갱신 함수에 의해 그 값이 달라질 수 있고, 상수의 값은 프로그램이 종료될 때까지 값이 달라지지 않는다는 점에서 둘은 대별된다.

그 다음 집중할 것은 `useEffect` 함수를 호출하는 대목이다. `useEffect`는 또 다른 함수 `f`와 배열 `[c]`를 인자로 받는 함수인데, 이때 배열은 상태 등의 반응형 *reactive* 값만으로 구성될 것이 강력히 권고된다 (Meta Platforms, Inc., 2026a). 상태 `s`와 상수 `c`는 모두 함수의 본문에 등장하지만 배열에는 `s`만이 존재하는데, 이는 `s`는 상태 변수로서 반응형 값이고 `c`는 상수로서 비반응형 값이기 때문이다.

여기서 몇 가지 의문이 남는다. 대형 언어 모델은 함수-배열 구조에서 배열에 어떤 항목이 올지 올바르게 예측할 수 있을까? 올바르게 예측할 수 있다면, 혹은 예측이 틀리다면, 모델은 어떤 정보를 근거로 삼아 토큰을 예측했을까? 모델이 리액트 라이브러리를 사용하여 코드가 작성되었다는 점을 인식하고 리액트 훅의 의미를 계산했기 때문일까, 아니면 함수와 배열이 매개변수로 쓰이는 문법 구조 때문일까? 그것도 아니라면, `useState` 같은 키워드를 단서로 삼는 것일까?

이 질문에 답하기 위해, 우리는 45개의 변수 쌍을 바탕으로 자바스크립트 코드 데이터셋을 구축하였다. 각 변수 쌍에 대해 동일한 코드 골격을 유지한 채 함수 이름, 변수 선언 형태, 변수 선언 순서, 변수 이름을 조작하여 40개의 변형 코드를 만들었다. 이 데이터를 이용하여 Llama 3.2 1B 모델과 Gemma 3 1B 모델이 배열의 첫 항목을 예측하도록 했고, 이때 예측되는 토큰의 로짓을 측정하였다. 이어서 함수 본문에서의 변수 사용과 배열을 도입하는 문맥을 각각 조작하는 두 건의 소거 *ablation* 실험을 통해 관찰된 선호가 무엇

으로 환원되는지를 분해하였다.

실험 결과, 우리는 다음 세 가지를 발견할 수 있었다.

- **상태 변수 선호.** 모델은 실험 조건 전반에서 배열의 첫 요소로서 상태 변수를 선호하는 모습을 보였다. 이 경향은 리액트 훅 `useEffect` 또는 `useLayoutEffect`가 사용되었는지, 일반 자바스크립트 함수 `subscribe`가 사용되었는지에 상관없이 일관되게 나타나며, 오히려 리액트 환경일 때보다 일반 자바스크립트 환경일 때 모델이 상태 변수를 선호하는 정도가 더 높았다.
- **선언 형태와 변수 사용 단서.** 이러한 모델의 선호는 두 가지 단서가 더해질 결과로 보인다. 하나는 변수가 `useState` 형태로 선언되었다는 ‘선언 형태’ 단서이고, 다른 하나는 그 변수가 함수 본문에서 실제로 쓰였다는 ‘본문 사용’ 단서이다. 두 단서는 대체로 합쳐지는 방향으로 작동하지만, 본문에서 상태 변수가 쓰이지 않은 상황에서 어느 단서가 더 강하게 작용하는지는 배열을 도입하는 문맥이 리액트 환경인지의 여부에 따라 달라진다.
- **배열로의 일반화.** 이 선호는 리액트의 의존성 배열에서만 나타나는 것이 아니다. `useEffect`나 `subscribe`처럼 함수와 배열을 인자로 전달하는 상황이 아니라 독립적인 배열을 완성할 때에도 똑같이 상태 변수를 더 선호한다. 따라서 모델이 상태 변수를 선호하는 현상은 함수-배열 패턴에서만 일어나는 것이 아니라 모델이 배열을 채우는 일반적 경향에 가깝다.

2 선행 연구

코드 모델의 표상 분석. 코드 모델이 코드의 어떤 정보를 학습하고 표상하는지를 분석한 연구는 여러 차례로 진행되어 왔다. Troshin and Chirkova(2022)은 사전 학습된 코드 모델의 표상이 문법 구조, 이름 *identifier* 및 이름공간 *namespace*, 데이터 흐름과 같은 정보를 얼마나 담고 있는지 분석하였다. Ma et al.(2024)은 이를 더 체계적으로 확장하여, 문법 구조, 제어 흐름 및 의존성, 데이터 의존성과 같은 코드 구조가 여러 코드 모델의 표상에 얼마나 반영되는지 살펴보았다. Maveli et al.(2025)은 기능적 동등

성 *functional equivalence* 평가를 통해, 코드 모델이 표면적 구문을 넘어 프로그램 의미를 얼마나 포착하는지 몰았다. 이들 연구는 코드 모델이 코드의 문법적·의미적 정보를 일부 표상한다는 배경을 제공하지만, 대체로 코드 전체나 코드 쌍, 전반적인 모델 표상처럼 거시적인 관점에서 분석을 진행하였다는 한계가 있다.

코드 모델의 표면 단서 민감성. 한편 코드 모델의 예측이 프로그램의 의미를 안정적으로 따르지 않는다는 점을 보인 연구도 있다. Yefet et al.(2020)은 작은 코드 변형만으로도 모델의 예측이 흔들릴 수 있음을 보였고, Wang et al.(2024)은 변수, 함수, 메서드 등의 이름이 코드 분석 과제의 성능에 상당한 영향을 줄 수 있음을 보였다. Orvalho and Kwiatkowska(2025)은 의미를 보존하는 변형 앞에서도 대형 언어 모델의 코드 이해와 예측이 불안정해질 수 있음을 밝혀냈다. 이들 연구는 코드 모델의 다음 토큰 예측이 코드의 표면적인 단서에 영향을 받을 가능성이 있음을 시사한다.

이처럼 기존 연구는 주로 완성된 코드의 정답 여부, 변형에 대한 강건성, 또는 모델 표상의 전반적 성질을 다루었다. 반면 이 연구는 모델의 행동적 선호를 국소적으로 관찰, 측정한다. 우리는 주어진 코드 조각의 다음에 이어질 토큰을 모델이 예측하도록 하고, 후보 토큰의 로짓 차이를 직접 비교하였다. 이를 통해 모델이 주어진 코드에서 어떤 행동을 취하는지, 또 어떤 표면 단서가 그 행동을 유발하는지를 밝히고자 한다.

3 리액트 훅

리액트 *React*는 웹 기반 사용자 인터페이스 *UI*를 개발하기 위한 자바스크립트 라이브러리이다. 리액트로 만들어진 *UI*는 컴포넌트 *component*의 트리로 구성되며, 각 컴포넌트는 화면의 한 조각과 그 동작을 선언적으로 기술하는 명세로, 부수효과 *side-effect*를 일으키지 않는 순수 함수의 형태이다. 리액트 런타임은 이 명세를 해석하기 위해 컴포넌트를 직접 호출하고, 컴포넌트는 화면에 무엇이 나타나야 하는지를 나타내는 자료구조를 반환한다. 이상적으로는, 같은 입력으로 호출된 컴포넌트는 같은 사용자 인터페이스를 반환해야 하지만, 실제 프로그램에서는 상태 변화나 사용자 상호작용, 네트워크 요청과 같은 부수효과도 함께 다루어야 한다(Lee et al., 2025).

훅 *Hook*은 함수형 컴포넌트 안에서 이러한 상태와 부수효과를 다룰 수 있게 해 주는 함수이다. 혹은 겉으로는 일반적인 함수 호출처럼 보이지만, 리액트 런타임과 연결되어 컴포넌트의 상태를 관리하거나 렌더링 이후 실행될 효과를 등록한다. 훅에는 여러 종류가 있지만, 이 연구에서는 *useState*와 *useEffect*, *useLayoutEffect*를 위주로 살펴본다.

useState. *useState*는 컴포넌트 안에서 상태를 관리하기 위한 훅이다. *useState*는 초기 상태 x_0 을 인자로 받아, 현재 상태를 나타내는 값 x 와 그 상태를 갱신하는 함수 x_{set} 의 순서쌍을 반환한다(Meta Platforms, Inc., 2026b). 주로 아래와 같이 구조 분해 할당 문법과 함께 쓰인다.

```
1 const [s, setS] = useState(0);
```

서론의 예시에서 본 s 가 바로 이런 방식으로 선언된 상태 변수이고, 이에 반해 $\text{const } c = 0$ 처럼 일반적인 방식으로 선언된 c 는 상수다. 이처럼 한 변수를 정의할 때는 *useState* 훅과 구조 분해 할당 문법, 갱신 함수의 존재를 동반하는 상태의 형태로도 선언할 수 있고, 일반 자바스크립트의 예약어인 *const*를 사용하여 상수의 형태로도 정의할 수 있다. 이 글에서는, 변수가 표면적으로 어떤 형태로 정의되었는지를 두고 ‘선언 형태’라고 부르기로 하자.

useEffect. *useEffect*는 특정 시점에 실행될 부수효과를 등록하기 위한 훅으로, 네트워크 요청이나 외부 시스템과의 동기화처럼 컴포넌트의 순수한 반환값만으로는 표현하기 어려운 동작을 기술하는 데 쓰인다. *useEffect* 훅은 콜백 함수 *callback* f 와 배열 $[e]$ 를 인자로 받으며, 화면이 그려지거나 *rendered* 의존성 배열에 있는 값이 변화할 때마다 리액트 런타임이 함수 f 를 실행한다(Meta Platforms, Inc., 2026c). 이때 f 를 효과 함수, $[e]$ 를 의존성 배열 *dependency array*이라고 부른다.

의존성 배열은 부수효과가 어떤 값의 변화에 반응해야 하는지를 표시하는 자리이므로, (Meta Platforms, Inc., 2026a)에서는 효과 함수 내부에서 참조되는 반응형 값만을 의존성 배열에 포함할 것을 권고한다. 반응형 값이란 컴포넌트 안에서 선언된 상태 등 프로그램을 실행하는 과정에서 변화할 수 있는 값을 의미한다. 반대 개념으로서 상수와 같은 비반응형 값이 있으며, 프로그램이 실행되는 동안 변화하지 않는 값을

이른다.

아래는 상태 `s`가 바뀌거나 화면이 그려질 때마다 `s`의 값을 콘솔에 출력하도록 하는 코드이다.

```
1 useEffect(() => {  
2   console.log(s);  
3 }, [s])
```

이처럼 `useEffect` 혹은 콜백 함수와 배열 인자를 외부 함수가 감싸는 형태로 쓰인다. 이 글에서는 이 문법 패턴을 ‘함수-배열 구조’라고 부른다.

useLayoutEffect. `useLayoutEffect`는 `useEffect`와 마찬가지로 부수효과를 등록하기 위한 훅이며, 효과 함수와 의존성 배열을 인자로 받는다는 점에서 동일한 함수-배열 구조를 가진다.

차이는 효과가 실행되는 시점에 있다. `useEffect`에 등록된 효과는 일반적으로 화면이 그려진 뒤 비동기적으로 실행되는 반면, `useLayoutEffect`에 등록된 효과는 브라우저가 화면을 다시 그리기 전에 동기적으로 실행된다(Meta Platforms, Inc., 2026d). 따라서 두 훅은 실행 시점과 사용 목적에서는 구별되지만, 이 연구의 관점에서는 모두 콜백 함수와 의존성 배열을 함께 받는 리액트 훅이라는 공통점을 가진다.

이 연구는 `useEffect`뿐 아니라 `useLayoutEffect` 조건도 함께 살펴봄으로써, 배열 첫 항목에 대한 모델의 선호가 특정 함수 이름 `useEffect`에만 묶인 것인지, 아니면 다른 `useEffect` 계열 훅에서도 유지되는지를 확인한다.

4 방법

4.1 과제

이 연구에서 다루는 과제는 ‘배열 완성 과제’로, 함수-배열 구조의 코드 조각이 주어졌을 때 이어지는 배열의 첫 위치에서 어떤 토큰이 높은 로짓을 받는지를 측정하는 것이다. 각 코드는 `function` 예약어로 시작하는 자바스크립트 함수 또는 리액트 컴포넌트이며, 함수 안에는 상태 변수와 상수가 함께 정의되어 있다. 모델에는 배열을 여는 [까지의 코드 조각이 주어지고, 그 다음 토큰 위치에서 상태 변수와 상수 각각에 부여된 로짓을 비교한다.

모델의 토큰 선호와 관련하여 이 연구가 검토하는 가설은 다음과 같다.

- **리액트 의미론 가설.** `useEffect`나 `useLayoutEffect` 같은 리액트 훅 문맥에서 상태 변수 선

호가 특히 강하게 나타날 것이다.

- **함수-배열 구조 가설.** 함수-배열 구조 일반에서 상태 변수 선호가 강하게 나타날 것이다.
- **선언 형태 가설.** 문법 패턴 및 의미론과 관계없이, `useState` 형태로 선언된 변수에 대한 일반적인 선호가 있을 것이다.
- **변수 사용 가설.** 문법 패턴 및 의미론과 관계없이, 함수 본문에서 실제로 사용된 변수에 대한 선호가 있을 것이다.

각 가설은 5장의 각 절에서 자세히 검토한다.

4.2 데이터셋

데이터셋은 45개의 변수 쌍을 바탕으로 구성된 자바스크립트 코드로 이루어진다. 각 변수 쌍에 대해 동일한 코드 골격을 유지한 채, 한 변수는 리액트 상태로, 다른 변수는 일반 상수로 정의되도록 만들었다. 각 코드는 두 후보 변수를 선언부에 먼저 도입한 뒤, 함수 본문 또는 그에 대응하는 코드 영역에서 변수를 사용하고, 마지막으로 배열의 첫 항목을 예측하게 하는 구조를 공유한다.

이 연구에서 사용한 데이터셋은 변수 쌍을 생성하고, 이 변수 쌍을 수동으로 작성한 코드 템플릿에 주입하여 조건별 프롬프트를 합성하는 방식으로 만들어졌다.

변수 쌍 생성. 다음으로 코드 템플릿에 채워넣을 변수 이름의 쌍을 생성하였다. 이 변수 쌍은 순서쌍 (x, y, α) 의 형태로 구성되어 있다. 이때 x 와 y 는 서로 다른 변수 이름, α 는 두 변수의 초깃값이다. Claude Sonnet 4.6(Anthropic, 2026)을 사용하여 50쌍의 변수 쌍을 생성하였으며, 이 작업에 사용한 프롬프트는 표 8에 있다.

아울러, 다음 기준에 따라 생성된 변수 쌍이 유효한지를 검증했다.

- x, y, α 가 Llama 3.2 1B 모델의 토큰나이저에서 각각 단일 토큰으로 처리되는가?
- x, y, α 가 Gemma 3 1B 모델의 토큰나이저에서 각각 단일 토큰으로 처리되는가?
- x, y 가 자바스크립트 또는 리액트에서 예약어로 사용되고 있는가?

검증 결과 50쌍 가운데 5쌍이 유효하지 않은 것으로 확인되어, 남은 45쌍만을 이후 실험에 사용하였다. 45쌍의 어휘 목록은 표 9에 있다.

코드 합성. 앞서 생성한 변수 쌍을 여러 코드 템플릿에 주입하여 표 1와 같은 실험 조건을 합성하였다.

첫째, 함수 이름 조작은 상태 변수 선회가 리액트 훅에 특수적인지 검증하기 위한 것으로, `useEffect`, `useLayoutEffect`, `alias`, `alias_ctrl`, `subscribe` 다섯 조건으로 구성된다. `useEffect`와 `useLayoutEffect`는 실제 리액트 훅이고, `subscribe`는 직접 작성한 자바스크립트 함수를 호출하는 대조군이다. `alias`는 `useEffect`를 다른 이름으로 별칭 호출하는 조건이며, `alias_ctrl`은 `subscribe`를 `alias`와 동일한 별칭으로 호출하는 대조 조건이다.

둘째, 변수 사용 소거는 선언 형태와 배열 도입 문맥을 고정한 채 함수 본문에서 어떤 변수가 쓰이는지를 `both`, `target_only`, `control_only`, `neither` 네 조건으로 조작한다. 각 조건에 대해서는 §5.2에서 자세히 설명한다.

셋째, 문맥 소거는 변수 선언과 본문을 고정한 채 배열을 도입하는 문맥을 `useEffect`, `subscribe`, `plain_array`, `return_array` 네 조건으로 조작한다. 각 조건에 대해서는 §5.3에서 자세히 설명한다.

조건별로 합성된 코드의 예시는 부록 A.1에 있다.

데이터셋 검증. 데이터셋 생성 후 구조 검증과 토큰 검증을 거쳤다. 측정 대상 변수 토큰과 배열을 닫는 `]`가 모든 조건에서 단일 토큰으로 정렬됨을 확인했으며, 검증에 실패했거나 제외된 변수 쌍은 없었다.

4.3 모델

이 연구는 Llama 3.2 1B(Meta, 2024)와 Gemma 3 1B(Team et al., 2025) 두 개의 인과적 대형 언어 모델을 사용한다. 두 모델 모두 자바스크립트 코드 조각을 입력으로 받아 이어지는 위치의 로짓을 산출하며, 우리는 이 로짓 가운데 배열의 첫 항목 위치에서 두 후보 변수에 부여된 값만을 비교한다.

두 모델은 구조 등이 서로 다르므로, 같은 조작에 대해 같은 방향의 반응이 반복되는지를 확인하는 데 유용하다. 다만 모델의 수와 크기는 제한적이므로, 이 글에서는 두 모델에서 공통으로 반복되는 부호와 대비를 중심으로 살펴본다.

4.4 측정

모든 프롬프트는 코드 앞부분만으로 구성되며, 의존성 배열을 여는 `[`에서 끝난다. 인과적 언어 모델은 입력의 각 위치에서 다음 토큰의 로짓을 산출하므로, 배열의 첫 항목, 즉 `[` 바로 다음 토큰에 대한 로짓은 이 `[` 위치에서 읽어낸다.

우리가 집중하는 것은 두 후보 변수 x , y 이다. 모델이 x 토큰과 y 토큰에 부여한 로짓을 각각 $\text{logit}(x)$, $\text{logit}(y)$ 라고 할 때, 두 후보의 로짓 차이 D 는 다음과 같이 계산할 수 있다.

$$D = \text{logit}(x) - \text{logit}(y)$$

이때 역할 교체의 여부에 따라 x 가 상태 변수이고 y 가 상수일 수도 있고, 반대로 x 가 상수이고 y 가 상태 변수일 수도 있다. 이 연구에서는 선언 형태에 따라 로짓 차이 D 의 부호를 정렬한 LD_{state} 를 주요 지표로 사용한다.

$$LD_{\text{state}} = \begin{cases} +D & x \text{가 상태일 때} \\ -D & x \text{가 상수일 때} \end{cases}$$

LD_{state} 가 양수면 상태 선회를, 음수는 상수 선회를 뜻한다.

모든 통계는 변수 쌍을 단위로 집계한 값에서 보고한다. 모델별·조건별 평균 LD_{state} , 양의 값을 보인 변수 쌍의 비율($r_{>0}$)과 코헨(Cohen)의 d 를 보고한다. p 값은 각 검정이 가정한 방향을 그대로 따르되, 단측 검정의 방향과 반대로 큰 효과가 나타난 경우에는 p 값을 효과 부재의 근거로 해석하지 않는다.

5 결과

5.1 선호 관찰

선호 관찰 실험에서는 함수-배열 구조에서 모델이 배열의 첫 항목으로 상태 변수를 상수보다 선호하는지 살펴본다. 실험 조건은 `useEffect`, `useLayoutEffect`, `alias`, `alias_ctrl`, `subscribe`로 구성되며, 총 1,800개의 코드 조각을 데이터로 사용한다. 여기서 확인하고자 하는 것은 모델이 상태 변수를 선호하는 경향이 리액트 훅 문맥에 특화되어 있는지, 아니면 일반 자바스크립트 함수 문맥에서도 같은 정도로 나타나는지이다.

	조작 축	조건	목적
공통	변수 역할	x 가 상태, y 가 상수 $\leftrightarrow$ y 가 상태, x 가 상수	이름 편향 통제
	선언 순서	선언부에서 상태 먼저 선언 $\leftrightarrow$ 상수 먼저 선언	위치 효과 통제
	사용 순서	함수 본문에서 x 먼저 사용 $\leftrightarrow$ y 먼저 사용	위치 효과 통제
실험별	함수 이름	useEffect, subscribe 등 5종	리액트 특정성 검증
	변수 사용	both, target_only 등 4종	변수 사용 단서 분해
	배열 문맥	useEffect, plain_array 등 4종	배열 완성 일반화 검증

Table 1: 데이터셋의 주요 조작 축. 함수 이름 조건은 선호 관찰(§5.1)에, 변수 사용 조건은 변수 사용 소거(§5.2)에, 배열 문맥 조건은 문맥 소거(§5.3)에 각각 대응함.

모델	조건	LD_{state}	d	$r_{>0}$
Llama 3.2 1B	useEffect	1.076	4.37	1.00
	useLayoutEffect	0.993	3.98	1.00
	alias	0.944	4.50	1.00
	alias_ctrl	1.251	5.47	1.00
	subscribe	1.318	5.17	1.00
Gemma 3 1B	useEffect	0.354	1.14	0.84
	useLayoutEffect	0.274	0.99	0.80
	alias	0.402	1.50	0.91
	alias_ctrl	0.641	1.90	0.98
	subscribe	0.750	2.00	0.98

Table 2: 선호 관찰 실험 결과(변수 쌍 단위 평균, $n = 45$). 양수는 역할 교체 재정렬 이후 상태 변수 선호를 뜻함. 굵게 표시한 행은 짝을 이루는 비리액트 대조군으로, 두 모델 모두에서 대응하는 리액트 혹은 조건보다 큰 값을 보임.

상태 선호의 관찰. 두 모델 모두 다섯 조건 전부에서 $LD_{state} > 0$ 을 보였다(표 2). Llama 3.2 1B는 alias의 0.944부터 subscribe의 1.318까지, Gemma 3 1B는 useLayoutEffect의 0.274부터 subscribe의 0.750까지 분포한다. $r_{>0}$ 은 Llama 3.2 1B에서 전부 1.00, Gemma 3 1B에서 0.80~0.98로, 모델이 상태 변수를 선호하는 경향은 조건에 관계없이 강하고 일관되게 나타난다.

상태 선호의 일반성. 그러나 리액트 혹은 실험군과 자바스크립트 대조군을 비교해 보면, 리액트 의미론 가설이 예측하는 방향과 반대의 결과가 나타났음을 알 수 있다. 같은 변수 쌍 안에서 subscribe 조건은 useEffect 조건보다 더 큰 LD_{state} 를 보였기 때문이다. 그 차이는 Llama 3.2 1B에서 평균 0.242($d = 1.17$, $p = 6.25 \times 10^{-10}$), Gemma 3 1B에서 평균 0.397($d = 2.59$, $p = 2.69 \times 10^{-21}$)이었다.

호출부의 표면 형태를 더 가깝게 맞춘 alias_ctrl과 alias의 대비에서도 같은 패턴이 나타났다. alias_ctrl은 alias보다 Llama 3.2 1B에서 평균 0.307($d = 2.22$, $p = 8.59 \times 10^{-19}$), Gemma 3 1B에서 평균 0.239($d = 2.30$, $p = 2.48 \times 10^{-19}$) 큰 LD_{state} 를 보였다. 상태 변수 선호가 리액트 특수한

것이라면 리액트 실험군에서 대응하는 자바스크립트 대조군보다 더 큰 LD_{state} 를 보여야 했으나, 실제로는 대조군의 LD_{state} 가 더 컸으므로, 모델의 상태 선호를 리액트 의미론의 영향으로 설명하기는 어렵다.

다른 리액트 혹은 useLayoutEffect에서도 상태 변수 선호는 나타났지만, useEffect보다 더 강하지는 않았다. 같은 변수 쌍 안에서 useLayoutEffect의 평균 LD_{state} 는 useEffect보다 Llama 3.2 1B에서 0.082, Gemma 3 1B에서 0.080 낮았다. 이 결과 역시 리액트 의미론 가설로 모델의 선호를 설명하기 어려움을 시사한다.

이상의 결과를 통해 모델의 상태 변수 선호가 안정적으로 관찰된다는 점, 그리고 이 선호가 리액트에 특화되어 있지 않다는 점을 알 수 있다.

5.2 변수 사용 소거

변수 사용 소거 실험은 선언 형태와 문맥을 고정하고, 함수 본문에서 어떤 변수가 사용되는지만 조작한다. 이 실험에서 조작하는 변수 사용 조건은 아래와 같다.

both x, y 모두 함수 본문에서 사용함. x 와 y 의 출현 순서는 ‘사용 순서’ 조작 축에 따름.

target_only x 만 함수 본문에서 사용함.

control_only y 만 함수 본문에서 사용함.

neither 두 변수 모두 함수 본문에서 사용하지 않음.

문맥은 `useEffect`와 `subscribe` 두 가지로 한정해 총 1,800개의 코드 조각을 사용한다. 이 실험의 목적은 상태 변수의 선언 자체에서 오는 선언 형태 단서와, 함수 본문에서 사용된 변수를 다시 불러오는 변수 사용 단서를 분리하는 데에 있다.

control_only. 가장 중요한 조건은 `control_only`이다. 이 조건에서는 상태 변수는 본문에서 쓰이지 않고 상수만 쓰이므로, 선언 형태 단서와 변수 사용 단서가 충돌한다. `control_only`에서 $LD_{state} > 0$ 이면 선언 형태 단서가 변수 사용 단서보다 우세하다는 뜻이고, 반대로 음수이면 변수 사용 단서가 더 우세하다는 뜻이다.

실제 `control_only`의 결과는 문맥에 따라 갈리는 모습이 관찰되었다(표 3). `useEffect` 문맥에서는 두 모델 모두 강한 음수를 보였다(Llama: -1.598 , $d = -3.41$; Gemma: -3.282 , $d = -5.45$). 이는 이 문맥에서 변수 사용 단서가 선언 형태 단서보다 더 강하게 작용했음을 의미한다. 반면 `subscribe` 문맥에서는 모델에 따라 결과가 달랐다. Llama 3.2 1B는 $+0.353(d = 0.82, r_{>0} = 0.82, p = 8.2 \times 10^{-7})$ 으로 선언 형태 단서가 남아 있었지만, Gemma 3 1B는 $-0.967(d = -1.78)$ 로 변수 사용 단서가 여전히 우세했다.

LD_{state} 의 순서. 다음으로 눈여겨볼 것은 네 가지 환경에서의 LD_{state} 값의 순서이다. 이 순서를 확인하기 위해 `both` 조건을 기준으로 삼았다. `both`에서는 두 후보 변수가 모두 함수 본문에 등장하므로, 변수 사용 단서가 어느 한쪽 후보만을 지지하지 않는다. 반면 `target_only`에서는 상태 변수만 본문에 등장하므로, 변수 사용 단서가 상태 변수 쪽으로 기울고, `control_only`에서는 상수만 본문에 등장하므로, 변수 사용 단서가 상수 쪽으로 기운다. 따라서 선언 형태 단서와 변수 사용 단서가 로짓 수준에서 더해지는 방식으로 작용한다면, 같은 변수 쌍 안에서 `target_only`는 `both`보다 큰 LD_{state} 를 보이고, `control_only`는 `both`보다 작은 LD_{state} 를 보여야 한다.

표 3으로부터 실제로 이 예측이 성립했다는 사실을 알 수 있다. Llama 3.2 1B의 `subscribe` 문맥에서

는 `target_only`가 `both`보다 평균 0.279 컸고, `control_only`는 `both`보다 평균 0.777 작았다. `useEffect` 문맥에서도 같은 패턴이 나타나, `target_only`는 `both`보다 평균 2.187 컸고, `control_only`는 평균 2.422 작았다. Gemma 3 1B에서도 `subscribe` 문맥에서는 각각 $+2.597$ 과 -1.713 , `useEffect` 문맥에서는 각각 $+5.568$ 과 -3.875 의 차이를 보였다. 이 차이는 모두 기대한 방향에서 통계적으로 유의했다($p < 10^{-7}$).

종합. 이상을 정리하면, `useEffect` 문맥에서는 두 모델 모두 변수 사용 단서가 선언 형태 단서보다 우세했다. `subscribe` 문맥에서는 `control_only`의 부호가 모델마다 갈리므로 선언 형태 단서가 남는 정도를 일률적으로 말하기 어렵다. 다만 두 모델 모두 `control_only` 조건에서는, `useEffect`보다 `subscribe` 문맥에서 더 큰 LD_{state} 를 보였다. 즉 리액트가 아닌 환경에서 선언 형태 단서가 더 많이 남으며, 이는 §5.1에서 관찰된 패턴과도 일치한다. 따라서 변수 사용 단서와 선언 형태 단서 중 어느 하나가 항상 지배적이지 않고, 배열을 도입하는 문맥에 따라 상대적 우세가 달라진다고 결론 지을 수 있다.

한편 `neither` 조건, 즉 두 후보 변수가 모두 본문에서 사용되지 않은 조건에서도 상태 변수 쪽으로 기울리는 경향이 있었다($r_{>0} 0.93 \sim 1.00$). 다만 이 조건은 보조적인 관찰로만 다루며, 이 연구의 핵심 주장은 `control_only`의 부호와 `target_only > both > control_only` 순서에 둔다.

5.3 문맥 소거

문맥 소거 실험은 변수 선언부와 함수 본문을 고정 한 채, 배열을 도입하는 문맥만 조작한다. 아래 조건에 따라 조작하여, 총 1,440개의 코드 조각을 실험에 사용하였다.

useEffect 리액트 혹은 `useEffect`에 콜백 함수와 배열을 인자로 전달하는 형태.

subscribe 일반 함수 `subscribe`에 콜백 함수와 배열을 인자로 전달하는 형태.

plain_array 콜백 함수를 외부 함수에 전달하지 않고, 두 후보 변수가 사용된 뒤 독립적인 배열 리터럴을 정의하는 형태.

return_array 콜백 함수를 외부 함수에 전달하지 않

모델	문맥	both	t/o	c/o	neither
Llama 3.2 1B	useEffect	0.824	3.010	-1.598	0.876
	subscribe	1.130	1.409	0.353	0.628
Gemma 3 1B	useEffect	0.593	6.160	-3.282	1.245
	subscribe	0.746	3.343	-0.967	1.197

Table 3: 변수 사용 소거 실험의 평균 LD_{state} (변수 쌍 단위, $n = 45$). t/o와 c/o는 각각 target_only와 control_only 조건을 뜻함. 네 가지 환경 모두에서 target_only > both > control_only 순서가 성립함.

고, 두 후보 변수가 사용된 뒤 배열 리터럴을 반환하는 형태.

이 실험의 목적은 상태 변수 선호가 useEffect나 subscribe처럼 콜백 함수와 배열이 외부 함수의 인자로 등장하는 함수-배열 구조에 묶여 있는지, 아니면 함수 호출문과 배열의 단순한 연쇄에서도 나타나는지 확인하는 데에 있다. 만약 상태 선호의 경향이 콜백 함수가 있는 useEffect와 subscribe에서만 관찰되고, 콜백 함수가 없는 plain_array와 return_array에서 사라진다면, 이 선호는 함수-배열 구조에 의존하는 현상으로 읽을 수 있다. 반대로 네 문맥 모두에서 상태 변수 선호 경향이 보인다면, 이 선호는 특정 함수-배열 구조에 국한되기보다 배열 완성 문맥 일반의 경향으로 해석할 수 있다.

LD_{state} 는 여덟 가지의 환경 전부에서 강한 양수였다(표 4). Llama 3.2 1B는 useEffect 1.290, subscribe 1.191, plain_array 1.428, return_array 2.966을, Gemma 3 1B는 useEffect 2.355, subscribe 2.229, plain_array 1.113, return_array 0.749를 보였다. 콜백 함수가 없는 plain_array와 return_array에서도 선호가 유지되므로, 이 현상은 의존성 배열이나 함수-배열 구조에만 국한되지 않고, 배열을 채울 때 나타나는 더 일반적인 경향에 가깝다고 보아야 한다.

다만 문맥별 효과 크기의 순서는 두 모델에서 일관되지 않았다. 같은 변수 쌍 안에서 return_array 조건과 useEffect 조건을 비교하면, Llama 3.2 1B에서는 return_array의 LD_{state} 가 useEffect보다 평균 1.676 컸지만($p = 7 \times 10^{-27}$), Gemma 3 1B에서는 오히려 평균 1.606 작았다($p = 4 \times 10^{-30}$). plain_array 조건도 같은 양상을 보였다. Llama 3.2 1B에서는 plain_array가 useEffect보다 평균 0.138 컸으나 이 차이는 유의하지 않았고($p = 0.069$), Gemma 3 1B에서는 평균 1.242 작았다($p = 9 \times 10^{-21}$). 즉 Llama 3.2 1B는 return_array에서 가장 큰 값을 보

이는 반면, Gemma 3 1B는 같은 조건에서 가장 작은 값을 보인다. 따라서 문맥에 따라 일관된 상태 선호의 크기 순서가 존재하는 것은 아니다.

이 절의 결과를 §5.1와 함께 보면, 상태 변수 선호가 나타나기 위해 리액트 특수성이나 함수-배열 구조가 반드시 필요한 것은 아닌 것으로 보인다. 리액트 혹은 아닌 함수 문맥에서도, 콜백 함수가 없는 문맥에서도 상태 변수 선호가 사라지지 않았다는 점은, 이 경향성이 특정 라이브러리나 특정 문법 구조에 갇힌 현상이라기보다 배열을 완성 과제에서 일어나는 일반적인 경향에 가깝다는 해석을 강화한다.

6 논의

세 실험의 결과는 한 방향으로 수렴한다. 선호 관찰에서는 자바스크립트 대조군이 대응하는 리액트 실험군보다 더 큰 LD_{state} 를 보였고, 변수 사용을 조작했을 때는 선언 형태 단서와 변수 사용 단서의 상대적 우세가 문맥에 따라 달라졌으며, 문맥을 조작했을 때는 함수-배열 구조가 아닌 문맥에서도 상태 변수 선호가 유지되었다. 이 세 결과를 종합하면, 관찰된 선호를 ‘모델이 리액트 혹은 의미를 이해했다’는 해석으로 보기보다는, 배열 완성 과제에서 useState 형태의 선언에 대한 단서와 함수 본문에서의 변수 사용 단서가 함께 작동한 결과로 해석할 수 있다.

이 해석은 코드 모델이 프로그램 의미뿐 아니라 표면적 단서에도 영향을 받을 수 있음을 보인 선행 연구와 방향을 같이한다(Yefet et al., 2020; Wang et al., 2024; Orvalho and Kwiatkowska, 2025). 다만 선행 연구에서는 코드의 표면적인 변화에 대한 모델 예측의 불안정성을 다루었다면, 이 연구에서는 배열을 완성하는 상황 속에서 useState 선언 패턴을 모델이 일관되게 선호하고 있음을 관찰하였다.

끝으로 이 연구의 범위를 밝힌다. 이 연구는 모델 내부의 어텐션 헤드 attention head나 회로를 식별한 연구가 아니며, 두 후보 토큰의 로짓 차이를 비교하고

모델	문맥	LD_{state}	d	$r_{>0}$
Llama 3.2 1B	useEffect	1.290	2.76	0.98
	subscribe	1.191	3.04	1.00
	plain_array	1.428	3.20	1.00
	return_array	2.966	6.78	1.00
Gemma 3 1B	useEffect	2.355	5.99	1.00
	subscribe	2.229	5.54	1.00
	plain_array	1.113	2.73	1.00
	return_array	0.749	2.50	1.00

Table 4: 문맥 소거 실험 결과(변수 쌍 단위 평균, $n = 45$). 여덟 가지 환경 전부에서 $LD_{state} > 0$ 임.

입력 조건을 소거하여 그 차이가 어떤 단서와 관련되는지를 관찰한 행동 분석 연구다. 따라서 어떤 내부 계산 경로가 선언 형태 단서나 변수 사용 단서를 구현하는지는 이 연구의 범위 밖에 있다. 이러한 해석가 능성 관점에서의 연구는 향후 과제로 남긴다(Miller et al., 2025; Rodriguez-Cardenas et al., 2023).

7 결론

이 연구는 Llama 3.2 1B와 Gemma 3 1B 모델을 대상으로, 배열의 첫 항목을 예측할 때 모델이 상태 변수를 상수보다 선호하는지, 그리고 그 선호가 어떤 단서에서 비롯되는지를 분석했다. 결과는 세 가지로 정리된다. 첫째, 모델은 배열의 첫 항목으로 상태 변수를 선호하며, 이 선호는 리액트 훅 문맥에 국한되지 않는다. 둘째, 이 선호는 변수가 `useState` 형태로 선언되었다는 선언 형태 단서와, 어떤 후보 변수가 본문에서 사용되었는지에 관한 변수 사용 단서가 결합한 결과로 보이며, 두 단서의 상대적 우세는 배열을 도입하는 문맥에 따라 달라진다. 셋째, 이 선호는 함수-배열 구조를 넘어 독립적인 배열 문맥에서도 나타난다.

이 발견은 함수 이름, 선언 형태, 변수 이름, 선언 순서, 변수 사용, 문맥을 체계적으로 조작함으로써, 관찰된 선호를 ‘모델이 리액트 의미를 이해했다’는 해석만으로 설명하기 어렵다는 점을 보여 준다. 오히려 이 결과는 배열 완성 과정에서 `useState` 형태의 선언에 대한 단서가 안정적으로 작동하며, 여기에 본문에서의 변수 사용 단서가 문맥에 따라 결합한다는 해석을 뒷받침한다.

다만 모든 주장은 로짓 수준의 행동 관찰에 머물며, 어떤 내부 계산 과정이 이 단서들을 구현하는지는 다루지 않았다. 특히 선언 형태 단서는 하나의 단서가 아니라 `useState` 키워드, 구조 분해 할당 패턴, 갱신 함수의 존재 등 여러 성분이 합쳐진 결과이다. 따라서

향후 연구에서는 선언 형태 단서를 더 세밀하게 분해하거나, 더 큰 모델과 다양한 모델 계열로 같은 조작을 확장하거나, 내부 계산 과정을 직접 분석하는 방향으로 이어질 수 있다.

한계

이 연구의 모든 측정은 코드의 앞부분만 주어진 상태에서 다음 토큰의 로짓을 비교하는 방식으로 이루어졌다. 따라서 실제 개발 환경처럼 모델이 여러 토큰을 순차적으로 생성하고, 이전 생성 결과가 이후 문맥에 다시 영향을 주는 상황까지 일반화할 수 있는지는 확인하지 않았다. 더 길고 상호작용적인 코드 완성 상황에서 같은 선호가 유지되는지는 후속 연구의 과제로 남는다.

이 연구는 Llama 3.2 1B와 Gemma 3 1B라는 1B 급 모델 두 종류만을 분석했다. 두 모델에서 같은 부호로 반복되는 패턴은 관찰되었지만, 이 결과가 더 큰 모델이나 다른 모델 계열로 그대로 일반화된다고 주장할 수는 없다. 더 다양한 규모와 계열의 모델에 같은 조작을 적용하는 것 역시 후속 과제다.

이 연구에서 상태 변수로 선언되었다는 조건은 `useState`라는 키워드의 존재, 배열을 구조 분해 할당하는 패턴, 갱신 함수의 존재 등 여러 성분이 얹힌 결과이다. 따라서 관찰된 선언 형태 단서가 이 세 성분 가운데 정확히 무엇에서 기인하는지는 분리하지 못했다. 이들 성분을 독립적으로 조작하는, 더 세밀한 소거 실험이 이를 분리하는 다음 단계가 될 수 있다.

마지막으로 이 연구는 후보 토큰의 로짓을 비교하고 소거 조건으로 그 차이를 분해한 행동 관찰 연구이며, 모델 내부의 어텐션 헤드 *attention head*나 회로를 직접 식별하지 않았다. 어떤 계산 경로가 선언 형태 단서와 변수 사용 단서를 구현하는지는 이 연구의 범위 밖에 있다. 활성화 패칭 *activation patching*이나 헤드

소거 *head ablation*와 같은 기법을 통한 해석가능성 연구도 향후 연구의 과제이다.

참고 문헌

- Anthropic. 2026. System card: Claude sonnet 4.6. <https://www-cdn.anthropic.com/bbd8ef16d70b7a1665f14f306ee88b53f686aa75/Claude%20Sonnet%204.6%20System%20Card.pdf>. Accessed: 2025-06-23.
- Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. *Program synthesis with large language models*. *CoRR*, abs/2108.07732.
- Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. *In-coder: A generative model for code infilling and synthesis*. In *The Eleventh International Conference on Learning Representations*.
- Jay Lee, Joongwon Ahn, and Kwangkeun Yi. 2025. *React-trace: A semantics for understanding react hooks: An operational semantics and a visualizer for clarifying react hooks*. *Proc. ACM Program. Lang.*, 9(OOPSLA2).
- Wei Ma, Shangqing Liu, Mengjie Zhao, Xiaofei Xie, Wenhan Wang, Qiang Hu, Jie Zhang, and Yang Liu. 2024. *Unveiling code pre-trained models: Investigating syntax and semantics capacities*. *ACM Transactions on Software Engineering and Methodology*.
- Nickil Maveli, Antonio Vergari, and Shay B. Cohen. 2025. *What can large language models capture about code functional equivalence?* In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6880–6918. Association for Computational Linguistics.
- Meta. 2024. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices>. Accessed: 2025-06-23.
- Meta Platforms, Inc. 2026a. Separating events from effects – react. <https://react.dev/learn/separating-events-from-effects>. Accessed: 2025-06-23.
- Meta Platforms, Inc. 2026b. State: A component’s memory – react. <https://react.dev/learn/state-a-components-memory>. Accessed: 2025-06-23.
- Meta Platforms, Inc. 2026c. Synchronizing with effects – react. <https://react.dev/learn/synchronizing-with-effects>. Accessed: 2025-06-23.
- Meta Platforms, Inc. 2026d. *useLayoutEffect* – react. <https://react.dev/reference/react/useLayoutEffect>. Accessed: 2025-06-23.

Samuel Miller, Daking Rai, and Ziyu Yao. 2025. *Mechanistic understanding of language models in syntactic code completion*. *Preprint*, arXiv:2502.18499.

Pedro Orvalho and Marta Kwiatkowska. 2025. *Are large language models robust in understanding code against semantics-preserving mutations?* *Preprint*, arXiv:2505.10443.

Daniel Rodriguez-Cardenas, David N. Palacio, Dipin Khati, Henry Burke, and Denys Poshyvanyk. 2023. Benchmarking causal study to interpret large language models for source code. In *2023 IEEE International Conference on Software Maintenance and Evolution*.

Stack Exchange, Inc. 2025. 2025 stack overflow developer survey. <https://survey.stackoverflow.co/2025/>. Accessed: 2026-06-23.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, and 197 others. 2025. *Gemma 3 technical report*. *Preprint*, arXiv:2503.19786.

Sergey Troshin and Nadezhda Chirkova. 2022. *Probing pretrained models of source codes*. In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 371–383. Association for Computational Linguistics.

Zhilong Wang, Lan Zhang, Chen Cao, Nanqing Luo, Xinzhi Luo, and Peng Liu. 2024. *How does naming affect llms on code analysis tasks?* *Preprint*, arXiv:2307.12488.

Noam Yefet, Uri Alon, and Eran Yahav. 2020. *Adversarial examples for models of code*. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30.

A 데이터셋 구축 상세

A.1 조건별 코드 예시

기본 데이터셋. 표 5는 다섯 개의 함수 이름 조작 조건에 대한 코드 예시를 보여준다. `pair_01` 어휘 쌍, 즉 $x = \text{page}$, $y = \text{ref}$, $\alpha = 1$ 을 적용하였으며, 변수의 역할 교체가 일어나지 않은 환경을 상정한 것이다. 역할 교체가 일어나는 경우 x 와 y 의 위치가 뒤바뀌어, x 가 상수, y 가 상태 변수가 된다.

소거 데이터셋. 표 6은 네 개의 변수 사용 조작 조건 `both`, `target_only`, `control_only`, `neither`에 대한 코드 예시를 보여준다. 변수 간 역할 교체가 없고, 상태 변수 다음에 상수가 선언되며, `useEffect` 문맥

조건	코드 예시
useEffect	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { fetch(`/api?x=\${page}&y=\${ref}`); }, [</pre>
useLayoutEffect	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useLayoutEffect(() => { fetch(`/api?x=\${page}&y=\${ref}`); }, [</pre>
alias	<pre>const myEffect = useEffect; function Component() { const [page, setPage] = useState(1); const ref = 1; myEffect(() => { fetch(`/api?x=\${page}&y=\${ref}`); }, [</pre>
alias_ctrl	<pre>function subscribe(callback, values) { callback(); } const myEffect = subscribe; function Component() { const [page, setPage] = useState(1); const ref = 1; myEffect(() => { fetch(`/api?x=\${page}&y=\${ref}`); }, [</pre>
subscribe	<pre>function subscribe(callback, values) { callback(); } function Component() { const [page, setPage] = useState(1); const ref = 1; subscribe(() => { fetch(`/api?x=\${page}&y=\${ref}`); }, [</pre>

Table 5: 함수 이름 조작 조건별 코드 예시. pair_01의 변수 이름 쌍을 적용한 결과임.

이 사용되는 환경의 코드 조각을 네 가지 조건에 따라 조작한 것이다. 변수 이름으로는 pair_01을 적용하였다.

표 7은 네 개의 문맥 조작 조건 useEffect, subscribe, plain_array, return_array에 대한 코드 예시를 보여준다. 변수 간 역할 교체가 없고, 상태 변수 다음에 상수가 선언되며, 상태 변수와 상수 모두 함수 본문에서 언급되는 환경의 코드 조각을 네 가지 조건에 따라 조작한 것이다. 변수 이름으로는 pair_01을 적용하였다.

A.2 변수 쌍 생성

표 8은 변수 이름 쌍 (x, y, α) 을 합성하기 위해 대형 언어 모델에 제공한 프롬프트이다. Claude Sonnet 4.6(Anthropic, 2026)이 이 프롬프트에 따라 작업을 수행하였으며, 총 50개의 변수 쌍을 생성하였다. 이후 변수명 제약 검증을 거쳐 유효한 45쌍의 변수명만을 실험에 사용하였다. 유효한 변수 쌍의 목록은 표 9에 있다.

조건	코드 예시
both	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { use(page, ref); }, [</pre>
target_only	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { use(page); }, [</pre>
control_only	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { use(ref); }, [</pre>
neither	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { use(); }, [</pre>

Table 6: 변수 사용 조작 조건별 코드 예시. pair_01의 변수 이름 쌍을 적용한 결과임.

조건	코드 예시
useEffect	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; useEffect(() => { use(page, ref); }, [</pre>
subscribe	<pre>function subscribe(callback, values) { callback(); } function Component() { const [page, setPage] = useState(1); const ref = 1; subscribe(() => { use(page, ref); }, [</pre>
plain_array	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; use(page, ref); const values = [</pre>
return_array	<pre>function Component() { const [page, setPage] = useState(1); const ref = 1; use(page, ref); return [</pre>

Table 7: 문맥 조작 조건별 코드 예시. pair_01의 변수 이름 쌍을 적용한 결과임.

프롬프트

You are generating variable name sets for a React useEffect dataset.

Template

```
```jsx
function Component() {
 const [{x}, set{x}] = useState({init});
 const {y} = {init};
 useEffect(() => {
 fetch(`/api?x=${{x}}&y=${{y}}`);
 }, [
```
```

Task

Generate exactly 50 unique tuples of (x, y, init).

Rules

x

- camelCase, single token
- Represents a React state variable – should read naturally as a piece of UI state
- Safe examples (confirmed single-token): `'name'`, `'count'`, `'page'`, `'query'`, `'title'`, `'color'`, `'limit'`, `'mode'`, `'index'`, `'status'`, `'id'`, `'key'`, `'flag'`, `'text'`, `'tag'`
- Avoid: `'userId'`, `'pageSize'`, `'isLoading'` – likely to split into multiple tokens

y

- camelCase, single token
- Represents a plain local const variable (not state) – same semantic domain as x but clearly a non-state name
- Must be different from x
- Safe examples: `'base'`, `'ref'`, `'size'`, `'max'`, `'min'`, `'cap'`, `'len'`, `'num'`, `'val'`, `'offset'`
- Avoid repeating x as y

init

- Integer literal or boolean literal, single token
- Integer examples: `'0'`, `'1'`, `'2'`, `'10'`, `'100'`
- Boolean examples: `'true'`, `'false'`

Output format

Return a JSON array of exactly 50 objects. No explanation, no markdown, no preamble.

```
[
  { "x": "count", "y": "base", "init": "0" },
  ...
]
```

Table 8: 변수 이름 쌍의 생성에 사용한 프롬프트

| # | x | y | α |
|----|--------|------|----------|
| 1 | page | ref | 1 |
| 2 | status | cap | false |
| 3 | mode | max | 0 |
| 4 | index | len | 0 |
| 5 | flag | min | true |
| 6 | tag | num | 0 |
| 7 | title | ref | 1 |
| 8 | color | val | 0 |
| 9 | text | base | false |
| 10 | id | num | 0 |
| 11 | key | max | 1 |
| 12 | name | len | 0 |
| 13 | count | val | 0 |
| 14 | mode | ref | false |
| 15 | page | min | 0 |
| 16 | query | base | 1 |
| 17 | index | cap | 0 |
| 18 | status | num | false |
| 19 | tag | len | 0 |
| 20 | title | val | 0 |
| 21 | color | max | 1 |
| 22 | flag | ref | true |
| 23 | text | min | 0 |
| 24 | id | cap | 0 |
| 25 | key | len | 1 |
| 26 | name | val | 0 |
| 27 | count | max | 0 |
| 28 | mode | num | false |
| 29 | flag | base | true |
| 30 | status | ref | false |
| 31 | text | cap | 0 |
| 32 | page | min | 1 |
| 33 | index | val | 0 |
| 34 | query | len | 0 |
| 35 | tag | max | 0 |
| 36 | color | base | 1 |
| 37 | title | num | 0 |
| 38 | key | ref | 0 |
| 39 | id | min | 0 |
| 40 | name | cap | 0 |
| 41 | mode | len | false |
| 42 | text | max | 0 |
| 43 | count | ref | 1 |
| 44 | status | base | false |
| 45 | flag | cap | true |

Table 9: 변수 쌍 45개의 전체 목록